

Why is `io.Reader` so weird?

Exploring the third best interface in Go

13 May 2025

Simon Law

Member of Technical Staff, Tailscale

error

```
package builtin

type error interface {
    Error() string
}
```

The error built-in interface type is the conventional interface for representing an error condition, with the nil value representing no error.

Stringer

```
package fmt

type Stringer interface {
    String() string
}
```

Stringer is implemented by any value that has a String method, which defines the “native” format for that value. The String method is used to print values passed as an operand to any format that accepts a string or to an unformatted printer such as Print.

io.Reader

```
package io

type Reader interface {
    Read(p []byte) (n int, err error)
}
```

Reader is the interface that wraps the basic Read method.

io.Reader

```
package io

type Reader interface {
    Read(p []byte) (n int, err error)
}
```

Reader is the interface that wraps the basic Read method.

Read reads up to `len(p)` bytes into `p`. It returns the number of bytes read ($0 \leq n \leq \text{len}(p)$) and any error encountered. Even if Read returns $n < \text{len}(p)$, it may use all of `p` as scratch space during the call. If some data is available but not `len(p)` bytes, Read conventionally returns what is available instead of waiting for more.

When Read encounters an error or end-of-file condition after successfully reading $n > 0$ bytes, it returns the number of bytes read. It may return the (non-nil) error from the same call or return the error (and $n == 0$) from a subsequent call. An instance of this general case is that a Reader returning a non-zero number of bytes at the end of the input stream may return either `err == EOF` or `err == nil`. The next Read should return 0, EOF.

Callers should always process the $n > 0$ bytes returned before considering the error `err`. Doing so correctly handles I/O errors that happen after reading some bytes and also both of the allowed EOF behaviors.

If `len(p) == 0`, Read should always return $n == 0$. It may return a non-nil error if some error condition is known, such as EOF.

Implementations of Read are discouraged from returning a zero byte count with a nil error, except when `len(p) == 0`. Callers should treat a return of 0 and nil as indicating that nothing happened; in particular it does not indicate EOF.

Implementations must not retain `p`.

io.ReadAll

```
package io

func ReadAll(r Reader) ([]byte, error)
```

`ReadAll` reads from `r` until an error or EOF and returns the data it read. A successful call returns `err == nil`, not `err == EOF`. Because `ReadAll` is defined to read from `src` until EOF, it does not treat an EOF from `Read` as an error to be reported.

```
r := strings.NewReader("Hello, 世界")

b, err := io.ReadAll(r)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("%s", b)
```

io.Reader

```
package io

type Reader interface {
    Read(p []byte) (n int, err error)
}
```

```
r := strings.NewReader("Hello, 世界")

b := make([]byte, 5)
for {
    n, err := r.Read(b)    // Read len(b) bytes at a time
    fmt.Printf("%s", b[:n]) // Process b before checking err
    if err != nil {
        if err == io.EOF { // io.EOF is a sentinel error
            break
        }
        log.Fatal(err)
    }
}
```

archive/tar.Reader

```
package tar

func (tr *Reader) Read(b []byte) (int, error) {
    if tr.err != nil { // Abort if the inner Reader returned an error.
        return 0, tr.err
    }
    n, err := tr.curr.Read(b) // Read from the inner Reader
    if err != nil && err != io.EOF {
        tr.err = err
    }
    return n, err
}
```


Standard Library

Read([]byte) in the Go standard library

Thank you

Simon Law

Member of Technical Staff, Tailscale

<https://tinyurl.com/tailcareers>